

Synthetic Motion Pretraining Improves Next-Step Real UAV Trajectory Prediction Across Neural Architectures

Yoav Raytsfeld, Alon Kipnis

AK:[Add affiliations]

Abstract—Accurate UAV trajectory prediction with neural sequence models is hampered by the scarcity of real labeled trajectory data. We investigate synthetic motion pretraining as a remedy: physically plausible UAV trajectories are generated through a closed-loop physics simulation and aligned with real optical-sensor measurements in a shared 3D unit-normalized directional representation. Three neural architectures—LSTM, S4, and Mamba—are evaluated under real-only training, zero-shot transfer, and synthetic pretraining followed by fine-tuning on real data. Across all architectures, synthetic pretraining consistently reduces held-out mean squared error (MSE), with relative improvements of 33–35% over real-only training that are statistically significant under the present evaluation protocol. We compare these architectures to a classical interacting multiple model (IMM) based on multiple extended Kalman filters. All synthetic pretrained neural architectures outperform IMM, whereas their real-only training versions underperformed IMM. Supporting analyses show that the benefit of transfer exceeds the performance spread across architectures, and that performance scales with both synthetic pretraining set size and effective real training pool size. These results demonstrate that physically plausible synthetic motion is an effective pretraining signal for next-step UAV trajectory prediction from direction-only measurements, and can substantially reduce reliance on scarce real labeled trajectory data in this setting.

I. INTRODUCTION

Accurate UAV trajectory prediction is important for surveillance, safety, autonomy, and downstream decision-making. Classical maneuvering-target tracking is rooted in recursive state estimation, from Kalman filtering to nonlinear formulations (extended Kalman filtering) with coordinated-turn models and interacting multiple model (IMM) filters that switch between candidate motion regimes [1–4]. UAV tracking studies in radar and surveillance contexts commonly adapt Kalman-filter variants and motion models to small, low-altitude targets, highlighting coordinate and model-selection sensitivities that also arise in our maneuvering setting [5]. In parallel, counter-UAS studies have shown that motion tracks themselves can carry useful discriminative information, including radar-track based separation of UAVs from birds [6, 7], as well as trajectory-movement-based and trajectory-characterization-based approaches for drone-vs-bird discrimination [8, 9].

Large-parameter, temporal neural-network models offer a contemporary alternative to hand-designed motion-model banks like IMM. Such alternatives include Long Short-Term Memory (LSTM) [10], structured state-space models such as S4 and its diagonal variant S4D [11, 12], as well as selective

state-space models such as Mamba [13]. The main obstacles to using these architectures for real-world UAV tracking are the scarcity of training data and domain shift—that is, changes in data properties between training and the operational environment. A common sim-to-real strategy is therefore to learn from synthetic data and adapt to real measurements through domain randomization, synthetic pretraining, or dynamics randomization [14–16]. Related UAV studies have shown that sim-to-real transfer can be effective in specific aerial autonomy settings, such as drone racing [17].

In this work, we study whether physically plausible synthetic motion can serve as an effective pretraining signal for next-step UAV trajectory prediction from optical sensor measurements. As range information is unavailable in such measurements, the data contain only the temporal direction of the observed object. This places the problem close to the classical bearings-only tracking regime, where observability and filter stability depend strongly on geometry and state representation [18, 19]. To generate synthetic training data, we do not use idealized geometric motion curves directly. Instead, we define reference trajectories and simulate their execution in closed loop through a flight-control and dynamics pipeline, so that the realized motion reflects physically plausible effects such as inertia, acceleration limits, and transient tracking errors. We then align these synthetic trajectories with real UAV track data in a shared 3D unit-normalized directional representation. Using this synthetic data, we pretrain each tracking model, then fine-tune and evaluate it on real UAV track data. We refer to this transfer learning setting as the sim-to-real scenario.

To assess the extent to which synthetic data improves performance, we compare each model in the sim-to-real scenario with the same model architectures trained solely on real data. Our results show that sim-to-real tracking MSE is consistently lower than the real-only MSE across LSTM, S4, and Mamba architectures. The gain between the scenarios is much larger than the performance gap among architectures themselves. We also compare those architectures to an IMM comprising a coordinated-turn EKF and an innovation-based adaptive constant-acceleration Kalman filter, with parameters tuned on the real-data folds. Interestingly, IMM outperforms LSTM, S4, and Mamba in the real-only scenario, but underperforms their synthetic pretrained version. We complement the comparison with zero-shot sim-to-real, in which the synthetic pretrained

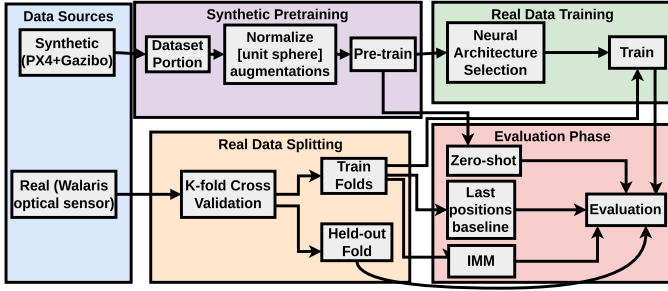


Fig. 1. **AK:[use vectorized graphic format]** Block diagram describing the sim-to-real UAV trajectory training and evaluation pipeline. Physically plausible UAV trajectories are generated in a PX4-based simulation environment, normalized to a shared 3D unit-normalized directional representation, and enriched through synthetic-side augmentations. Optically-measured azimuth and elevation of real maneuvering UAV are aligned to the same representation. Neural models are pretrained on different portions of the synthetic dataset in a cross-validation fashion. The resulting framework supports architecture selection, model training, zero-shot transfer, and evaluation of the classical IMM estimator and the trivial last-position baseline on held-out real data.



Fig. 2. **AK:[use vectorized graphic format]** Closed-loop synthetic trajectory generation pipeline. A reference motion pattern is first defined as a kinematic curve and sampled into position setpoints. These setpoints are sent to the PX4 autopilot, which computes control commands to track the desired motion. Gazebo then propagates the vehicle state according to the physical dynamics and returns state feedback to the controller. The resulting executed trajectory is logged and used as the synthetic training trajectory.

model is evaluated on real test data without fine-tuning on real data.

II. METHOD

A. Problem Formulation

We study next-step UAV trajectory prediction from vision-based directional measurements. Because the real signal provides bearing information but not range, different objects along the same line of sight can map to the same coordinate. This ambiguity makes prediction under limited real data especially challenging.

To align synthetic and real domains, both are mapped into a shared 3D unit-normalized directional representation, referred to below as a unit-sphere representation. Each input sequence consists of a fixed window of $W = 48$ timesteps with 7-dimensional features

$$\mathbf{x}_t = [x_t, y_t, z_t, dx_t, dy_t, dz_t, dt_t]. \quad (1)$$

The prediction target is the next 3D displacement in normalized space,

$$\Delta \mathbf{p}_{t+1} \in \mathbb{R}^3. \quad (2)$$

B. Synthetic Trajectory Generation

To obtain a large and diverse training set, we generate synthetic UAV trajectories procedurally. Rather than using idealized kinematic curves directly as training data, we simulate their physical execution in a closed loop, so that the resulting

trajectories reflect inertia, acceleration limits, and transient tracking errors.

The generation process follows a closed-loop reference-tracking pipeline, illustrated in Fig. 2. First, a reference trajectory is defined geometrically, using either parametric motion equations or waypoint-generation rules, and sampled at a fixed time step of $\Delta t = 0.03$ s. These sampled positions are then provided as position setpoints to a simulated flight controller. The controller tracks the commanded motion through cascaded feedback loops: an outer position loop generates velocity commands, and an inner velocity loop converts them into low-level actuation-related commands such as thrust and attitude targets. A physics simulator then propagates the vehicle state under platform dynamics and physical constraints. The realized motion produced by this interaction is logged as the synthetic trajectory.

In our implementation, this pipeline is realized in the PX4 software-in-the-loop (SITL) environment [20]. PX4 provides the autopilot and setpoint-tracking logic, while Gazebo simulates the vehicle dynamics and physical environment [21]. Offboard reference commands are communicated to the autopilot through the MAVLink protocol [22]. As a result, the synthetic data correspond to executed closed-loop trajectories rather than ideal reference curves. To increase motion diversity, the reference generator includes random waypoints, squares, figure-8s, helices, racetracks, climb-waves, side-steps, U-turns, snakes, and spirals.

C. Representation and Normalization

The synthetic data are normalized to match the native real-data representation. For the reported one-step experiments, robust dataset-level normalization is computed from the training split only, using median and interquartile range (IQR) statistics. These statistics are then reused during evaluation. In the transfer setting, the pretraining statistics are preserved during fine-tuning and evaluation to maintain alignment between the synthetic and real domains.

In the final reported one-step experiments, no online augmentation is applied to the real dataset. Synthetic-side augmentation is used only during pretraining to enrich the simulated trajectories while preserving the unit-sphere structure.

III. EXPERIMENTAL SETUP

A. Datasets and Splits

The real dataset used in this study is drawn from Walaris optical tracking logs acquired by a stationary camera system. Each file records timestamped observations at roughly 25 Hz and includes a 3D unit direction vector toward the tracked object together with additional sensor metadata. Although the source dataset contains multiple object categories, we retain only UAV recordings in this work, yielding 53 real trajectories in total. This curated real-UAV subset will be released upon publication of the paper. All splits are created at the trajectory or file level rather than the frame level, which avoids leakage when multiple derived samples originate from

the same recording. We evaluate using trajectory-level cross-validation with $K \in \{2, 5, 10\}$, with validation data sampled solely from the training pool. These protocols expose the models to progressively larger effective real-data pools while preserving held-out trajectories for testing.

The synthetic dataset used in this study comprises 439 trajectories spanning 7 classes of flying platforms, including fixed-wing aircraft, VTOL platforms, and multirotor UAVs, and 10 maneuver families **AK:[How much time the trajectories cover? what is the sampling resolution?]**. Because both synthetic and real trajectories are represented in the same shared direction-only state space, the role of the synthetic corpus is to provide broad airborne motion priors rather than object-specific appearance cues. For synthetic pretraining, it is divided into training, validation, and test splits before downstream transfer to the real dataset.

B. Models and Comparison Methods

We compare LSTM, S4, and Mamba under a controlled setup in which all models use the same 48-step input history, shared normalized representation, prediction target, optimizer, and loss. Each model maps the 7D input to a 128-dimensional hidden layer, applies a four-layer sequence backbone, and predicts the next 3D displacement from the final hidden state. The backbones differ only in the sequence encoder, namely a four-layer LSTM, a four-layer diagonal S4D encoder, or four stacked Mamba blocks.

As a classical comparator, we evaluate a two-branch interacting multiple model (IMM) filter consisting of a 7D coordinated-turn EKF with state $[x, y, z, v, \text{heading}, \text{turn_rate}, v_z]$ and a 9D innovation-based adaptive constant-acceleration Kalman filter with state $[x, y, z, v_x, v_y, v_z, a_x, a_y, a_z]$. In both branches, the measurement model is position-only, so each update receives only the 3D observation $[x, y, z]$. Initialization is performed separately for each sliding window from the first one or two observations, with turn rate and accelerations initialized to zero and covariance set to $10I$. For each fold, branch-specific process and measurement noise statistics are estimated from the training split, while transition, mode-probability, innovation-adaptation, and Q/R scaling parameters are selected on the validation split. For fairness, IMM is evaluated on the same 48-step history as the neural models. We also report a trivial last-position baseline.

C. Training Protocols

We consider two main training regimes: real-only training from scratch and synthetic pretraining followed by fine-tuning on real data. We also evaluate zero-shot transfer from synthetic pretraining and analyze how downstream performance changes with the amount of synthetic data used during pretraining.

All reported one-step neural experiments are trained with AdamW using a learning rate of 10^{-5} , weight decay of 10^{-3} , a batch size of 128, and a maximum of 50 epochs, with early stopping patience of 5. The objective is a weighted Huber loss with $\delta = 1.0$.

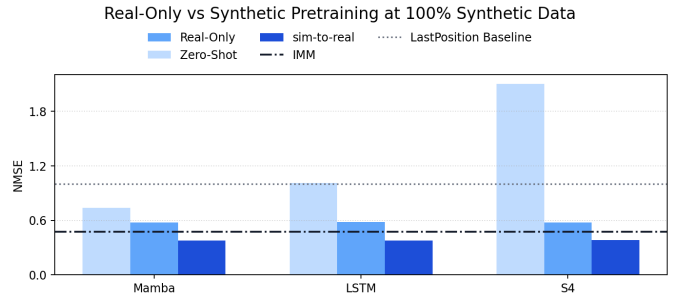


Fig. 3. **AK:[Does IMM operates in a zero-shot fashion? remove the 'at 100 % synthetic Data' from the title. In fact, you can remove the title since the caption provides all the information. the same applies for other figures.]** Prediction MSE over real test data of three neural prediction architectures (Mamba, LSTM, S4) under different training scenarios (real-only, sim-to-real, zero-shot). NMSE is the MSE normalized by last-position predictor baseline, defined in (3). All neural architectures outperforms IMM only under sim-to-real.

TABLE I

NMSE ($\pm$ SE) FOR EACH ARCHITECTURE UNDER REAL-ONLY TRAINING (R) AND SYNTHETIC PRETRAINING FOLLOWED BY FINE-TUNING (S2R), WITH RELATIVE GAIN AND p -VALUES FOR PAIRED IMPROVEMENT TESTS AGAINST REAL-ONLY ($P_{R \rightarrow S2R}$) AND AGAINST IMM ($P_{IMM \rightarrow S2R}$).

Model	R	S2R	Gain (%)	$P_{R \rightarrow S2R}$	$P_{IMM \rightarrow S2R}$
LSTM	0.582 ± 0.054	0.377 ± 0.062	35.2	7.85×10^{-5}	6.93×10^{-4}
Mamba	0.577 ± 0.051	0.379 ± 0.063	34.4	1.55×10^{-4}	1.07×10^{-3}
S4	0.576 ± 0.051	0.385 ± 0.056	33.3	9.25×10^{-5}	8.31×10^{-3}
IMM	0.477 ± 0.078	—	—	—	—

To study the effect of the synthetic pretraining scale, each neural architecture is pretrained on four fractions of the synthetic training split: 10%, 25%, 50%, and 100%.

D. Metrics and Statistics

All results are evaluated on held-out trajectories using an online one-step prediction protocol. At time $t-1$, the predictor receives past observations up to time $t-1$ and estimates the next state $\hat{\mathbf{p}}_t$. We quantify performance using the Normalized Mean Squared Error (NMSE), defined as

$$\text{NMSE} = \frac{\sum_{t=2}^T \|\mathbf{p}_t - \hat{\mathbf{p}}_t\|_2^2}{\sum_{t=2}^T \|\mathbf{p}_t - \mathbf{p}_{t-1}\|_2^2} \quad (3)$$

The denominator corresponds to the cumulative squared error of a zero-order-hold (ZOH) predictor, which uses the previous observation as the prediction for the next state. Thus, $\text{NMSE} < 1$ indicates that the model outperforms this naive baseline. Table I shows the mean NMSE for each model and training regime across the 10 held-out test folds. The accompanying uncertainty is reported as the standard error across folds, computed as $\text{SE} = s/\sqrt{10}$, where s is the sample standard deviation of the 10 fold-level NMSE values. Paired t -tests are performed on the 10 matched fold pairs for both the real-only versus transferred comparison and the transferred versus IMM comparison, shown in Table I.

IV. RESULTS AND DISCUSSION

A. Main Transfer Results

Synthetic pretraining followed by fine-tuning consistently improves held-out performance relative to real-only training across all three neural architectures. Figure 3 and Table I summarize the primary $k = 10$ comparison.

The improvement is consistent across LSTM, Mamba, and S4. Relative to real-only training, synthetic pretraining followed by fine-tuning yields relative NMSE improvements of 35.2%, 34.4%, and 33.3%, respectively. These gains are statistically significant for all three models under the present cross-validation protocol, supporting a consistent transfer advantage in this dataset.

The comparison to the classical IMM estimator further strengthens this conclusion. Under the primary reported setting, all three transferred neural models significantly outperform IMM, showing that synthetic pretraining can close and reverse the gap between classical filtering and learned sequence models in this direction-only tracking setting. The last-position baseline is also an important reference, because a learned tracking model that does not outperform it would have limited practical value. In our experiments, all transferred neural models outperform the last-position baseline, whereas zero-shot transfer does not. This indicates that synthetic motion alone is not sufficient, and that adaptation to real data remains necessary.

Taken together, these results suggest that the main conclusion is not which neural architecture performs best, since the differences among LSTM, Mamba, and S4 are comparatively small, but that all three benefit substantially from transfer. In this setting, initialization from synthetic motion appears to matter more than the specific architecture choice.

B. Effect of Synthetic Pretraining and Real-Data Quantity

Figures 4 and 5 provide supporting scale analyses. Figure 4 shows that even modest synthetic pretraining improves fine-tuned performance relative to the real-only setting, with the largest gains appearing between 0% and 10% pretraining, followed by more gradual improvement as the synthetic fraction increases. At the full synthetic fraction, all transferred neural models outperform the classical IMM estimator.

Figure 5 shows a complementary trend across effective real training pools of roughly 50%, 80%, and 90% of the dataset. Larger real-data pools are generally associated with lower NMSE for both the neural models and IMM. Since these settings also induce different train-test partitions, we treat this comparison as supporting evidence rather than as the primary benchmark. Overall, the clearest and most stable result is that downstream performance benefits from introducing synthetic pretraining, with additional gains from increasing both the synthetic pretraining scale and the effective amount of real training data.

V. CONCLUSION

We presented a sim-to-real framework for next-step UAV trajectory prediction from direction-only measurements. The

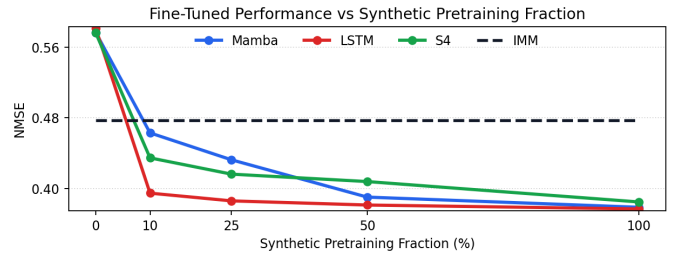


Fig. 4. AK:[Change 'Synthetic Pretraining Fraction' to 'Fraction of Synthetic Pretraining Data'. Also, remove the title. Can you show shaded areas for SE?] Synthetic data improves real prediction. Normalized Mean Squared Error (NMSE) as a function of the amount of synthetic pretraining data for all prediction model architectures. IMM (dashed) is not pretrained on synthetic data. All other models are fine-tuned on real data after synthetic pretraining. AK:[Can you add text below 0 says 'real-only'?] The point at 0% corresponds to real-only training without synthetic pretraining.

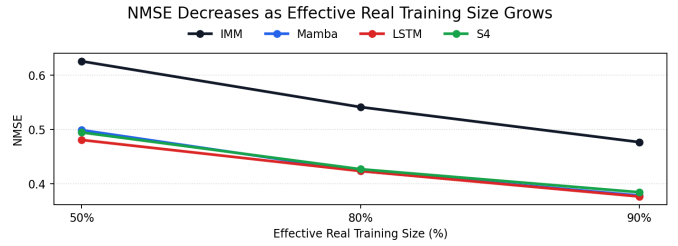


Fig. 5. AK:[Change 'Effective Real Training Size' to 'Fraction of Real Training Data'. Also, remove the title. Can you show shaded areas for SE?] Amount of real training data versus prediction error. Normalize Mean Squared Error (NMSE) of all prediction architectures as a function of the fraction of training data. The three settings correspond to effective training pools of roughly 50%, 80%, and 90% of the dataset, associated with 26–27, 42–43, and 47–48 trajectories, respectively.

framework creates physically plausible synthetic trajectories with a shared 3D unit-normalized directional representation. Prediction models are pre-trained on the synthetic trajectories before fine-tuned on real UAV direction measurements.

We compared three neural architectures (LSTM, Mamba, S4) trained under this framework to a classical IMM trained under a real-only scenario. Across all neural architectures, sim-to-real consistently improved held-out tracking performance relative to real-only training. Supporting analyses further indicate that performance benefits from both increasing the scale of synthetic pretraining and increasing the effective amount of real training data. The results show that physically plausible synthetic trajectory generation can provide an effective pre-training signal for next-step UAV trajectory prediction while reducing dependence on scarce labeled real trajectory data in this setting.

REFERENCES

- [1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960. [Online]. Available: <https://doi.org/10.1115/1.3662552>
- [2] M. Roth, G. Hendeby, and F. Gustafsson, "Ekf/ukf maneuvering target tracking using coordinated turn

- models with polar/cartesian velocity,” in *17th International Conference on Information Fusion (FUSION)*, 2014, pp. 1–8. [Online]. Available: <https://www.hendeby.se/publication.html>
- [3] H. A. P. Blom and Y. Bar-Shalom, “The interacting multiple model algorithm for systems with markovian switching coefficients,” *IEEE Transactions on Automatic Control*, vol. 33, no. 8, pp. 780–783, 1988. [Online]. Available: <https://doi.org/10.1109/9.1299>
 - [4] X. R. Li and V. P. Jilkov, “Survey of maneuvering target tracking. part i: Dynamic models,” *IEEE Transactions on Aerospace and Electronic Systems*, vol. 39, no. 4, pp. 1333–1364, 2003. [Online]. Available: <https://doi.org/10.1109/TAES.2003.1261132>
 - [5] Y. Wei, T. Hong, and M. Kadoch, “Improved kalman filter variants for uav tracking with radar motion models,” *Electronics*, vol. 9, no. 5, p. 768, 2020. [Online]. Available: <https://doi.org/10.3390/electronics9050768>
 - [6] W. S. Chen, J. Liu, and J. Li, “Classification of uav and bird target in low-altitude airspace with surveillance radar data,” *The Aeronautical Journal*, vol. 123, no. 1260, pp. 191–211, 2019. [Online]. Available: <https://doi.org/10.1017/aer.2018.158>
 - [7] D. Tahmoush, “Uav discrimination from birds using radar track information,” in *Radar Sensor Technology XXVII*, ser. Proceedings of SPIE, vol. 12535, 2023, p. 125350X. [Online]. Available: <https://doi.org/10.1117/12.2666281>
 - [8] A. Nesteruk, V. Nikitin, Y. Albrekht, L. Scislo, D. Grela, and P. Krol, “Distinguishing a drone from birds based on trajectory movement and deep learning,” *Sensors*, vol. 26, no. 3, p. 755, 2026. [Online]. Available: <https://doi.org/10.3390/s26030755>
 - [9] S. Srigrarom, K. H. Chew, D. M. D. Lee, and P. Ratsamee, “Drone versus bird flights: Classification by trajectories characterization,” in *2020 59th Annual Conference of the Society of Instrument and Control Engineers of Japan (SICE)*, 2020. [Online]. Available: <https://doi.org/10.23919/SICE48898.2020.9240313>
 - [10] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
 - [11] A. Gu, K. Goel, and C. Ré, “Efficiently modeling long sequences with structured state spaces,” in *The Tenth International Conference on Learning Representations (ICLR)*, 2022, oral. [Online]. Available: <https://openreview.net/forum?id=uYLFoz1v1AC>
 - [12] A. Gu, K. Goel, A. Gupta, and C. Ré, “On the parameterization and initialization of diagonal state space models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 35 971–35 983. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/e9a32fade47b906de908431991440f7c-Abstract-Conference.html
 - [13] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.00752>
 - [14] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” *arXiv preprint arXiv:1703.06907*, 2017. [Online]. Available: <https://arxiv.org/abs/1703.06907>
 - [15] J. Tremblay, A. Prakash, D. Acuna, M. Brophy, V. Jampani, C. Anil, T. To, E. Cameracci, S. Boochoon, and S. Birchfield, “Training deep networks with synthetic data: Bridging the reality gap by domain randomization,” in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2018, pp. 969–977. [Online]. Available: <https://doi.org/10.1109/CVPRW.2018.00143>
 - [16] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 3803–3810. [Online]. Available: <https://doi.org/10.1109/ICRA.2018.8460528>
 - [17] A. Loquercio, E. Kaufmann, R. Ranftl, A. Dosovitskiy, V. Koltun, and D. Scaramuzza, “Deep drone racing: From simulation to reality with domain randomization,” *IEEE Transactions on Robotics*, vol. 36, no. 1, pp. 1–14, 2020. [Online]. Available: <https://doi.org/10.1109/TRO.2019.2942989>
 - [18] V. J. Aidala and S. E. Hammel, “Utilization of modified polar coordinates for bearings-only tracking,” *IEEE Transactions on Automatic Control*, vol. 28, no. 3, pp. 283–294, 1983. [Online]. Available: <https://doi.org/10.1109/TAC.1983.1103230>
 - [19] S. C. Nardone, A. G. Lindgren, and K. F. Gong, “Fundamental properties and performance of conventional bearings-only target motion analysis,” *IEEE Transactions on Automatic Control*, vol. 29, no. 9, pp. 775–787, 1984. [Online]. Available: <https://doi.org/10.1109/TAC.1984.1103664>
 - [20] L. Meier, D. Honegger, and M. Pollefeys, “Px4: A node-based multithreaded open source robotics framework for deeply embedded platforms,” in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 6235–6240. [Online]. Available: <https://doi.org/10.1109/ICRA.2015.7140074>
 - [21] N. Koenig and A. Howard, “Design and use paradigms for gazebo, an open-source multi-robot simulator,” in *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, vol. 3, 2004, pp. 2149–2154. [Online]. Available: <https://doi.org/10.1109/IROS.2004.1389727>
 - [22] A. Koubaa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, and M. Khalgui, “Micro air vehicle link (mavlink) in a nutshell: A survey,” *IEEE Access*, vol. 7, pp. 87 658–87 680, 2019. [Online]. Available: <https://doi.org/10.1109/ACCESS.2019.2924410>